The Journal of Open Source Software

molecular-simulations: Reproducible computational biophysics workflows and analysis

Matt Sinclair[1], **Moeen Meigooni**[1], **Archit Vasan**[1], **and Arvind Ramanathan**[1,2]

1 Argonne National Laboratory **2** University of Chicago

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Christoph Junghans

Reviewers:

- @jglaser
- @daico007

Submitted: 15 July 2026
Published: unpublished

Summary

molecular-simulations is a Python toolkit for building, running, and analyzing molecular dynamics (MD) simulations using the AMBER force-field ecosystem with OpenMM as the simulation engine, and Parsl as the workflow layer for scalable execution on HPC systems. The package provides end-to-end components that reduce the friction of going from an input structure to AMBER-compatible system files, GPU-accelerated MD with restartable protocols, and analysis routines commonly used in characterizing biomolecular simulations. By covering the full arc from preparation through analysis behind a single Python API, it lets researchers treat a simulation campaign as one reproducible program rather than a chain of hand-managed scripts.

The package is organized around three stages that mirror the lifecycle of an MD study (Figure 1):

- **Build.** System preparation via AmberTools for explicit- and implicit-solvent setups (default ff19SB protein parameters and OPC water) (Izadi et al., 2014; Tian et al., 2020), small-molecule parameterization with GAFF2 and AM1-BCC charges (Jakalian et al., 2002; Wang et al., 2004), complex assembly, constant-pH-ready builds, and coarse-grained construction for the CALVADOS model (Bülow et al., 2025).
- **Simulate.** OpenMM-based drivers standardizing equilibration and production protocols with checkpoint/restart, straightforward injection of custom forces, constant-pH MD, Empirical Valence Bond (EVB) free-energy calculations (Warshel & Weiss, 1980), umbrella sampling with MBAR/WHAM free-energy estimation (Kumar et al., 1992; Shirts & Chodera, 2008), and coarse-grained/multi-resolution simulation.
- **Analyze.** Automated routines for conformational clustering, protein–protein interaction characterization, interaction-energy fingerprinting, covariance-based attractive/repulsive interaction detection (Golcuk & Gur, 2025), solvent-accessible surface area via the Shrake–Rupley algorithm (Shrake & Rupley, 1973), parallel MM-PBSA binding free energies (Miller III et al., 2012), and interface scoring — ipTM (Evans et al., 2021; Zhang & Skolnick, 2004), ipSAE (Dunbrack Jr, 2025), pDockQ (Bryant et al., 2022), pDockQ2 (Zhu et al., 2023), and LIS (Kim et al., 2024).

The package also ships agent-loadable *skills* — self-contained tool descriptions, one per stage — so it can be driven directly by language-model agents and tool-calling frameworks without a bespoke wrapper.

Statement of need

Modern biomolecular MD projects frequently require *ensembles* of simulations across multiple systems (e.g., homologs, mutants, complexes, or replicate trajectories) coupled to custom analyses. In practice, this workflow often becomes a collection of bespoke scripts spanning

structure cleanup, force-field assignment, solvation and ion placement, simulation protocol execution, checkpoint/restart management, and post-processing. Each of these steps carries its own conventions and failure modes, and the glue between them is rarely captured anywhere durable. This ad hoc approach makes it difficult to scale from a single workstation run to a production campaign on an HPC cluster or cloud resource while preserving reproducibility and provenance.

The problem is compounded by the layer at which most researchers work. Preparation is typically driven through AmberTools' interactive utilities, execution through hand-written OpenMM scripts, and analysis through a separate stack (MDAnalysis, cpptraj, in-house code). Reproducing a result months later, or handing a campaign to a collaborator, then depends on tacit knowledge of which script was run in which order with which arguments. As datasets and the number of systems under study grow, this friction can become the dominant cost of a project rather than the science itself.

molecular-simulations addresses this need by providing:

1. **System building utilities** that automate common AMBER/AmberTools preparation steps (including explicit and implicit solvent workflows, complex assembly, ligand parameterization, and constant-pH-ready builds).
2. **Simulation drivers** built on OpenMM that standardize equilibration/production protocols with restart support and enable easy integration of custom forces and free-energy methods.
3. **HPC-ready execution** using Parsl, including reusable configuration objects for local execution and PBS-style clusters, so the same code scales from a laptop to a supercomputer.
4. **Integrated analysis** routines focused on clustering and protein–protein interaction characterization, including energetic and structural metrics frequently used to rank and interpret interfaces.

Because every stage is expressed in a single Python API, an entire campaign — build, simulate, analyze — can be written, version-controlled, and re-executed as one program, improving both reproducibility and provenance relative to a directory of loosely coupled scripts.

State of the field

Several software packages address aspects of MD workflow management and HPC execution.

For GROMACS users, `gmxapi` provides a Python interface enabling programmatic control of simulations, including custom stopping conditions and user plugins within force calculations (Irrgang et al., 2018; Irrgang & Davis, 2022). However, `gmxapi` operates within a single job allocation, limiting its applicability to multi-node campaigns spanning multiple scheduler submissions. The recently published `asyncmd` package (Jung & Hummer, 2025) extends GROMACS workflows across job boundaries using Python's async/await syntax, with a focus on providing building blocks for trajectory-based enhanced sampling algorithms such as transition path sampling and weighted ensemble methods. While `asyncmd` excels at dynamic stopping conditions and adaptive sampling workflows, it currently supports only the SLURM scheduler and does not include system preparation or post-simulation analysis capabilities.

General-purpose workflow frameworks such as AiiDA (Huber et al., 2020) and signac/row (Adorf et al., 2018) enable reproducible computational workflows with automatic provenance tracking. These tools prioritize data management over MD-specific functionality; for instance, AiiDA lacks native support for GPU resource requests, and MD integration requires additional plugins.

molecular-simulations occupies a complementary niche by providing an end-to-end toolkit spanning system preparation through analysis, with a focus on the AMBER/OpenMM ecosystem. Unlike orchestration-focused tools, it includes system builders (explicit/implicit solvent, ligand parameterization), standardized simulation protocols with custom force injection, and integrated

analysis routines (clustering, MM-PBSA, interaction fingerprinting, interface scoring). The use of Parsl for workflow execution provides scheduler-agnostic HPC support (SLURM, PBS, AWS, Google Cloud, and others) while maintaining a simple Python API, enabling users to scale from local prototyping to production campaigns with minimal code changes.

None of the tools above delivers the specific combination this package targets: AMBER/OpenMM system preparation, GPU MD with in-process custom-force and free-energy support, and the analysis routines needed to interpret an interaction, all callable from one Python API and portable across schedulers. Composing existing tools to reach that combination would mean gluing together four separate contracts and failure modes — precisely the ad hoc pipeline described above — so we built a cohesive toolkit instead; the scholarly contribution is the integrated, reproducible arc from structure to interpreted result, together with analyses (e.g., per-residue interaction-energy decomposition in OpenMM) that are otherwise not available out of the box.

Software design

The central design decision is to separate the lifecycle into three loosely coupled stages — build, simulate, analyze — that communicate through files in standard formats (AMBER topologies/coordinates, OpenMM checkpoints, trajectories) rather than through shared in-memory state.

OpenMM was chosen as the simulation engine specifically because it is performant, Python-native, and enables easy extension and adoption of new algorithms and techniques. This makes it possible to inject custom forces, per-residue energy decomposition, constant-pH moves, and EVB/umbrella-sampling terms in-process. AMBER/AmberTools handles preparation for the same reason of leverage — we standardize and script community-validated tooling rather than reimplement force-field assignment and solvation.

For execution we adopt Parsl instead of a bespoke scheduler wrapper or a single-allocation model. Resource specifications live in reusable configuration objects that are decoupled from the scientific code, so the same build/simulate/analyze program runs unchanged on a laptop or across many PBS/SLURM/cloud allocations. The simulation drivers deliberately ship opinionated equilibration/production protocols with checkpoint/restart as defaults, while preserving adapters (custom force injection, overridable protocol steps) — trading maximal configurability for reproducible, correct-by-default behavior that a non-specialist can run and a specialist can extend. This is not meant to replace OpenMM: for complex, bespoke simulations it remains best practice to write one's own protocol, which can still benefit from our analysis suite and Parsl distribution framework.

Anticipating that agentic frameworks and tool-calling interfaces are becoming a standard way to drive scientific software, the package also bundles a set of *skills* — self-contained `SKILL.md` documents (one per build, simulate, analyze, and HPC-deployment task) that an agent loads on demand, sharing the same stage boundaries and one contract with the human-facing Python API. Combined with the grounded protocols and Parsl configuration objects, this lowers the expertise barrier substantially: an agent equipped with the skills can turn a natural-language request into a correct build→simulate→analyze campaign across HPC allocations without hand-written `tleap` inputs, OpenMM protocols, or scheduler scripts — the mechanism behind its integration into StructBioReasoner (see Research impact statement).

Research impact statement

molecular-simulations is already in external use as the simulation and analysis backbone of StructBioReasoner, an agentic framework for designing biologics that target intrinsically disordered proteins (Sinclair et al., 2026). That integration relies on the package's single-

API coverage and Parsl-based scalability to let autonomous agents build, run, and interpret simulations without hand-managed scripting — a concrete demonstration that the design supports programmatic, campaign-scale use beyond its original authors.

For near-term significance, the package ships reproducibility infrastructure and a worked validation example rather than a new method. Its constant-pH support adapts the established discrete-protonation-state Monte Carlo algorithm of Swails et al. (Swails et al., 2014), reusing the openmm-cph implementation and wiring it into the package's build/simulate/analyze workflow; the contribution here is the packaging and a bundled, reproducible benchmark, not the underlying technique. That benchmark (`scripts/benchmark_hewl.py`) titrates the acidic and histidine sites of hen egg-white lysozyme against experimental NMR pKa values (Bartik et al., 1994; Webb et al., 2011) (Figure 2), giving users a regression target they can reproduce on their own hardware. Community-readiness is supported by public distribution on PyPI, hosted documentation on Read the Docs, a continuous-integration test suite, and an OSI-approved (MIT) license, lowering the barrier for external adoption and contribution.

Figures

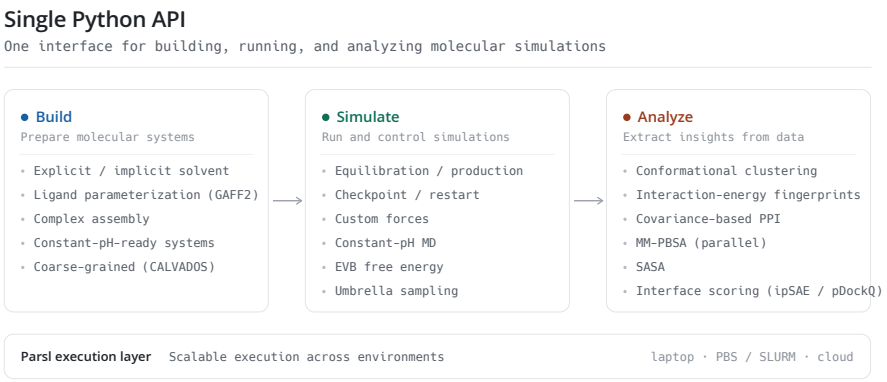

Figure 1: Architecture of *molecular-simulations*. The toolkit is organized into three stages — build, simulate, and analyze — exposed through a single Python API and executed locally or across HPC/cloud resources via Parsl.

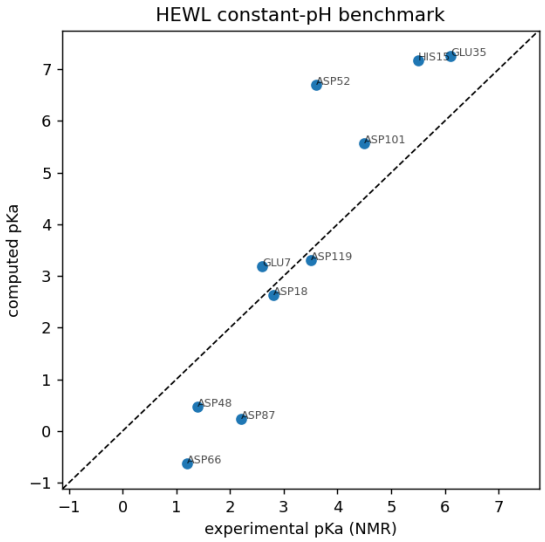

Figure 2: Hen egg-white lysozyme constant-pH benchmark: computed side-chain pKa values versus experimental NMR references, produced by the bundled `benchmark_hewl.py` harness.

AI usage disclosure

Anthropic's Claude (Opus 4 through 4.8) was used in the generation of documentation, the CI pipeline, unit-test scaffolding, and code refactoring. In addition to code generation, Opus 4.8 was used in the editing of this text. All AI generated code and text has been reviewed and validated by the authors.

Acknowledgements

molecular-simulations builds on the open-source scientific Python ecosystem and depends on: AmberTools for system preparation, OpenMM for the simulation engine, Parsl for scalable workflow execution, MDAnalysis and MDTraj for trajectory analysis, scikit-learn for clustering, and NumPy and SciPy for the underlying numerical routines. (Babuji et al., 2019; Case & others, 2023; Eastman et al., 2024; Gowers et al., 2016; Harris et al., 2020; McGibbon et al., 2015; Michaud-Agrawal et al., 2011; Pedregosa et al., 2011; Virtanen et al., 2020)

References

Adorf, C. S., Dodd, P. M., Ramasubramani, V., & Glotzer, S. C. (2018). Simple data and workflow management with the signac framework. *Computational Materials Science*, *146*, 220–229. https://doi.org/10.1016/j.commatsci.2018.01.035

Babuji, Y., Woodard, A., Li, Z., Katz, D. S., Clifford, B., Kumar, R., Lacinski, L., Chard, K., Wozniak, J. M., & Foster, I. (2019). Parsl: Pervasive parallel programming in python. *Proceedings of the 28th International Symposium on High-Performance Parallel and Distributed Computing (HPDC '19)*, 25–36. https://doi.org/10.1145/3307681.3325400

Bartik, K., Redfield, C., & Dobson, C. M. (1994). Measurement of the individual pKa values of acidic residues of hen and turkey lysozymes by two-dimensional 1H NMR. *Biophysical Journal*, *66*(4), 1180–1184. https://doi.org/10.1016/S0006-3495(94)80900-2

Bryant, P., Pozzati, G., & Elofsson, A. (2022). Improved prediction of protein-protein

interactions using AlphaFold2. *Nature Communications*, *13*(1), 1265. https://doi.org/10.1038/s41467-022-28865-w

Bülow, S. von, Yasuda, I., Cao, F., Schulze, T. K., Trolle, A. I., Rauh, A. S., Crehuet, R., Lindorff-Larsen, K., & Tesei, G. (2025). Software package for simulations using the coarse-grained CALVADOS model. *arXiv Preprint arXiv:2504.10408*.

Case, D. A., & others. (2023). AmberTools. *Journal of Chemical Information and Modeling*, *63*(20), 6183–6191. https://doi.org/10.1021/acs.jcim.3c01153

Dunbrack Jr, R. L. (2025). Rēs ipSAE loquunt: What's wrong with AlphaFold's ipTM score and how to fix it. *bioRxiv*.

Eastman, P., Galvelis, R., Peláez, R. P., Abreu, C. R. A., Farr, S. E., Gallicchio, E., Gorenko, A., Henry, M. M., Hu, F., Huang, J., Krämer, A., Michel, J., Mitchell, J. A., Pande, V. S., Rodrigues, J. P. G. L. M., Rodriguez-Guerra, J., Simmonett, A. C., Singh, S., Swails, J., … Markland, T. E. (2024). OpenMM 8: Molecular dynamics simulation with machine learning potentials. *The Journal of Physical Chemistry B*, *128*(1), 109–116. https://doi.org/10.1021/acs.jpcb.3c06662

Evans, R., O'Neill, M., Pritzel, A., Antropova, N., Senior, A., Green, T., Žídek, A., Bates, R., Blackwell, S., Yim, J., Ronneberger, O., Bodenstein, S., Zielinski, M., Bridgland, A., Potapenko, A., Cowie, A., Tunyasuvunakool, K., Jain, R., Clancy, E., … Hassabis, D. (2021). Protein complex prediction with AlphaFold-multimer. *bioRxiv*. https://doi.org/10.1101/2021.10.04.463034

Golcuk, M., & Gur, M. (2025). A practical covariance-based method for efficient detection of protein–protein attractive and repulsive interactions in molecular dynamics simulations. *Journal of Chemical Information and Modeling*, *65*(19), 9865–9870.

Gowers, R. J., Linke, M., Barnoud, J., Reddy, T. J. E., Melo, M. N., Seyler, S. L., Domański, J., Dotson, D. L., Buchoux, S., Kenney, I. M., & Beckstein, O. (2016). MDAnalysis: A python package for the rapid analysis of molecular dynamics simulations. *Proceedings of the 15th Python in Science Conference (SciPy 2016)*, 98–105. https://doi.org/10.25080/Majora-629e541a-00e

Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., Kerkwijk, M. H. van, Brett, M., Haldane, A., Río, J. F. del, Wiebe, M., Peterson, P., … Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, *585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Huber, S. P., Zoupanos, S., Uhrin, M., Talirz, L., Kahle, L., Häuselmann, R., Gresch, D., Müller, T., Yakutovich, A. V., Andersen, C. W., Ramirez, F. F., Adorf, C. S., Gargiulo, F., Kumbhar, S., Passaro, E., Johnston, C., Merkys, A., Cepellotti, A., Mounet, N., & Pizzi, G. (2020). AiiDA 1.0, a scalable computational infrastructure for automated reproducible workflows and data provenance. *Scientific Data*, *7*(1), 300. https://doi.org/10.1038/s41597-020-00638-4

Irrgang, M. E., & Davis, C. A. K. (2022). Gmxapi: A GROMACS-native python interface for molecular dynamics with ensemble and plugin support. *PLOS Computational Biology*, *18*(2), e1009835. https://doi.org/10.1371/journal.pcbi.1009835

Irrgang, M. E., Hays, J. M., & Kasson, P. M. (2018). Gmxapi: A high-level interface for advanced control and extension of molecular dynamics simulations. *Bioinformatics*, *34*(22), 3945–3947. https://doi.org/10.1093/bioinformatics/bty484

Izadi, S., Anandakrishnan, R., & Onufriev, A. V. (2014). Building water models: A different approach. *The Journal of Physical Chemistry Letters*, *5*(21), 3863–3871. https://doi.org/10.1021/jz501780a

Jakalian, A., Jack, D. B., & Bayly, C. I. (2002). Fast, efficient generation of high-quality atomic

charges. AM1-BCC model: II. Parameterization and validation. *Journal of Computational Chemistry*, *23*(16), 1623–1641. https://doi.org/10.1002/jcc.10128

Jung, H., & Hummer, G. (2025). Asyncmd: A python library to orchestrate complex molecular dynamics simulation campaigns on high performance computing systems. *Journal of Open Source Software*, *10*(112), 8321. https://doi.org/10.21105/joss.08321

Kim, A.-R., Hu, Y., Comjean, A., Rodiger, J., Mohr, S. E., & Perrimon, N. (2024). Enhanced protein-protein interaction discovery via AlphaFold-multimer. *bioRxiv*. https://doi.org/10.1101/2024.02.19.580970

Kumar, S., Rosenberg, J. M., Bouzida, D., Swendsen, R. H., & Kollman, P. A. (1992). The weighted histogram analysis method for free-energy calculations on biomolecules. I. The method. *Journal of Computational Chemistry*, *13*(8), 1011–1021. https://doi.org/10.1002/jcc.540130812

McGibbon, R. T., Beauchamp, K. A., Harrigan, M. P., Klein, C., Swails, J. M., Hernández, C. X., Schwantes, C. R., Wang, L.-P., Lane, T. J., & Pande, V. S. (2015). MDTraj: A modern open library for the analysis of molecular dynamics trajectories. *Biophysical Journal*, *109*(8), 1528–1532. https://doi.org/10.1016/j.bpj.2015.08.015

Michaud-Agrawal, N., Denning, E. J., Woolf, T. B., & Beckstein, O. (2011). MDAnalysis: A toolkit for the analysis of molecular dynamics simulations. *Journal of Computational Chemistry*, *32*(10), 2319–2327. https://doi.org/10.1002/jcc.21787

Miller III, B. R., McGee Jr., T. D., Swails, J. M., Homeyer, N., Gohlke, H., & Roitberg, A. E. (2012). MMPBSA.py: An efficient program for end-state free energy calculations. *Journal of Chemical Theory and Computation*, *8*(9), 3314–3321. https://doi.org/10.1021/ct300418h

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, *12*, 2825–2830.

Shirts, M. R., & Chodera, J. D. (2008). Statistically optimal analysis of samples from multiple equilibrium states. *The Journal of Chemical Physics*, *129*(12), 124105. https://doi.org/10.1063/1.2978177

Shrake, A., & Rupley, J. A. (1973). Environment and exposure to solvent of protein atoms. Lysozyme and insulin. *Journal of Molecular Biology*, *79*(2), 351–371. https://doi.org/10.1016/0022-2836(73)90011-9

Sinclair, M., Meigooni, M., Vasan, A., Gokdemir, O., Lian, X., Ma, H., Babuji, Y., Brace, A., Hossain, K., Siebenschuh, C., & others. (2026). Scalable agentic reasoning for designing biologics targeting intrinsically disordered proteins. *Platform for Advanced Scientific Computing Conference*, 1–13.

Swails, J. M., York, D. M., & Roitberg, A. E. (2014). Constant pH replica exchange molecular dynamics in explicit solvent using discrete protonation states: Implementation, testing, and validation. *Journal of Chemical Theory and Computation*, *10*(3), 1341–1352. https://doi.org/10.1021/ct401042b

Tian, C., Kasavajhala, K., Belfon, K. A. A., Raguette, L., Huang, H., Migues, A. N., Bickel, J., Wang, Y., Pincay, J., Wu, Q., & Simmerling, C. (2020). ff19SB: Amino-acid-specific protein backbone parameters trained against quantum mechanics energy surfaces in solution. *Journal of Chemical Theory and Computation*, *16*(1), 528–552. https://doi.org/10.1021/acs.jctc.9b00591

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., Walt, S. J. van der, Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E.,

... Mulbregt, P. van. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods*, *17*(3), 261–272. https://doi.org/10.1038/s41592-019-0686-2

Wang, J., Wolf, R. M., Caldwell, J. W., Kollman, P. A., & Case, D. A. (2004). Development and testing of a general amber force field. *Journal of Computational Chemistry*, *25*(9), 1157–1174. https://doi.org/10.1002/jcc.20035

Warshel, A., & Weiss, R. M. (1980). An empirical valence bond approach for comparing reactions in solutions and in enzymes. *Journal of the American Chemical Society*, *102*(20), 6218–6226. https://doi.org/10.1021/ja00540a008

Webb, H., Tynan-Connolly, B. M., Lee, G. M., Farrell, D., O'Meara, F., Sondergaard, C. R., Teilum, K., Hewage, C., McIntosh, L. P., & Nielsen, J. E. (2011). Remeasuring HEWL pKa values by NMR spectroscopy: Methods, analysis, accuracy, and implications for theoretical pKa calculations. *Proteins: Structure, Function, and Bioinformatics*, *79*(3), 685–702. https://doi.org/10.1002/prot.22886

Zhang, Y., & Skolnick, J. (2004). Scoring function for automated assessment of protein structure template quality. *Proteins: Structure, Function, and Bioinformatics*, *57*(4), 702–710. https://doi.org/10.1002/prot.20264

Zhu, W., Shenoy, A., Kundrotas, P., & Elofsson, A. (2023). Evaluation of AlphaFold-multimer prediction on multi-chain protein complexes. *Bioinformatics*, *39*(7), btad424. https://doi.org/10.1093/bioinformatics/btad424